

T⁴: Test-Time Training for Taste

Learning Research Reasoning Through Synthetic Literature Review

Sachin Konan

1 Motivation

A researcher’s taste—their ability to identify which problems are worth solving—is shaped by a massive, heterogeneous stream of information: reading papers, attending talks, watching lectures, conversations with collaborators, browsing Twitter threads, reviewing code repositories. From all of this, the best researchers extract signal: they notice gaps between lines of work, judge which gaps matter, and connect ideas that others haven’t linked. The result is a research direction that feels inevitable in retrospect but required consuming and synthesizing enormous context to identify.

Can I build models that do this? The vision is a language model that consumes massive streams of scientific information and asks the right questions—not just answers them. Today, standard pretraining learns $P(\text{paper} \mid \text{literature})$, the distribution over published outcomes, but this captures only the artifact. The intermediate reasoning—why an author read 40 papers, dismissed most, noticed a specific gap between two, and judged it worth three years of work—is invisible in the training data.

Recent work on synthetic pretraining suggests this is tractable. Synthesizing diverse representations of knowledge via cross-document relationships consistently outperforms naive training on raw text: Yang et al. (2024) demonstrate that *knowledge rearrangement*—generating varied representations of the same facts through entity-graph traversal—boosts QA accuracy from 39.5% to 56.2% where naive continued pretraining actually *hurts* performance. Yang et al. (2025) scale this principle to general pretraining by explicitly modeling inter-document conditionals $P(d_2 \mid d_1)$, recovering 58% of a $20\times$ -data oracle at 1T-token scale. Crucially, Kim et al. (2025) show that *how* synthetic data is organized matters as much as its content: stitching related generations into long megadocuments and inserting latent-thought rationales between segments yields $1.80\times$ data efficiency, with benefits substantially larger on long-context evaluation. Meanwhile, Dong et al. (2024) demonstrate that scientific evolution itself has learnable structure—chronological BERT models on 1.75M arXiv papers achieve 0.991 AUC on citation prediction and reveal smooth temporal trajectories in weight space, suggesting the patterns underlying scientific progress are consistent enough to model.

The gap: these methods improve knowledge acquisition or predict citations, but none capture the intermediate reasoning of literature review—the thought process where a researcher reads broadly, selectively attends to what matters, identifies gaps, and formulates questions. I want to synthesize *that process* as training data, and train models that can consume long, noisy streams of scientific content and extract the right questions from them.

2 Approach: AutoReview Pipeline

I reconstruct research reasoning trajectories from arXiv and assemble them into long-context synthetic training data in three stages.

Stage 1: LLM-Guided Citation Crawl. I use the Kaggle arXiv metadata corpus for title→ID resolution and download per-paper LaTeX source from `arxiv.org/src/{id}`, extracting structured references via BibTeX/BBL parsing. The crawl is BFS over the citation graph: at each node, an LLM ranks references by importance with rationale, and I follow the top- K that have arXiv source. This produces a citation tree where edges carry explicit signals about *why* to follow each citation.

Stage 2: Synthetic Annotation. For each paper in the tree, I run a sequence of LLM calls that build on each other, with the full reading history as context. Concretely, for paper i in the traversal:

1. **Summary** (~ 200 tokens): the LLM receives the paper’s abstract and full text (or a compressed version for long papers) and produces a structured summary covering the core problem, method, and key results.
2. **Gap analysis** (~ 300 tokens): the LLM receives the summary *plus all prior summaries and gap analyses from papers* $1 \dots i-1$. It identifies open questions, unstated assumptions, and limitations—specifically those that become visible in the context of what has already been read. This is the key difference from summarizing papers independently: the accumulated context lets the LLM notice gaps *between* papers, not just within them.
3. **Cross-paper connections** (every 2–3 papers, ~ 400 tokens): the LLM receives all accumulated annotations so far and generates explicit connections—e.g., “Paper 3’s linear-time primitive could address the efficiency bottleneck identified in Paper 1’s gap analysis, but would need Paper 2’s approximation guarantees.” These

are the synthetic analogs of the connections a researcher makes while reading.

4. **Research idea** (final paper only, ~ 500 tokens): after the full traversal, the LLM receives all annotations and generates a concrete research direction: a problem statement, proposed method, and expected contribution. This is the *target* for training. I construct these by using the actual paper at the root of the citation tree as the ground truth—the idea that the author actually pursued after doing this literature review.

Each call uses the OpenAI-compatible chat API (served via vLLM), with the accumulated annotations concatenated into the system/user prompt. I use a 70B+ instruction-tuned model (e.g., Llama 3.1 70B) for annotation quality. The total annotation cost per paper is ~ 1400 generated tokens; for a crawl of 500 papers, this is $\sim 700K$ tokens of synthetic annotations.

Stage 3: Long-Context Megadoc Assembly. Following the megadoc principle (Kim et al., 2025), I interleave paper content with synthetic annotations into long training sequences. Figure 1 illustrates the structure. Each paper is followed by its annotations (summary, gaps), and after every few papers I insert cross-paper reasoning that connects ideas across the reading history. The sequence terminates with the target research idea. This mirrors the “latent thoughts” approach from Kim et al. (2025)—inserting reasoning between content segments—but the thoughts here are research-specific: gap analyses and cross-paper connections rather than generic rationales.

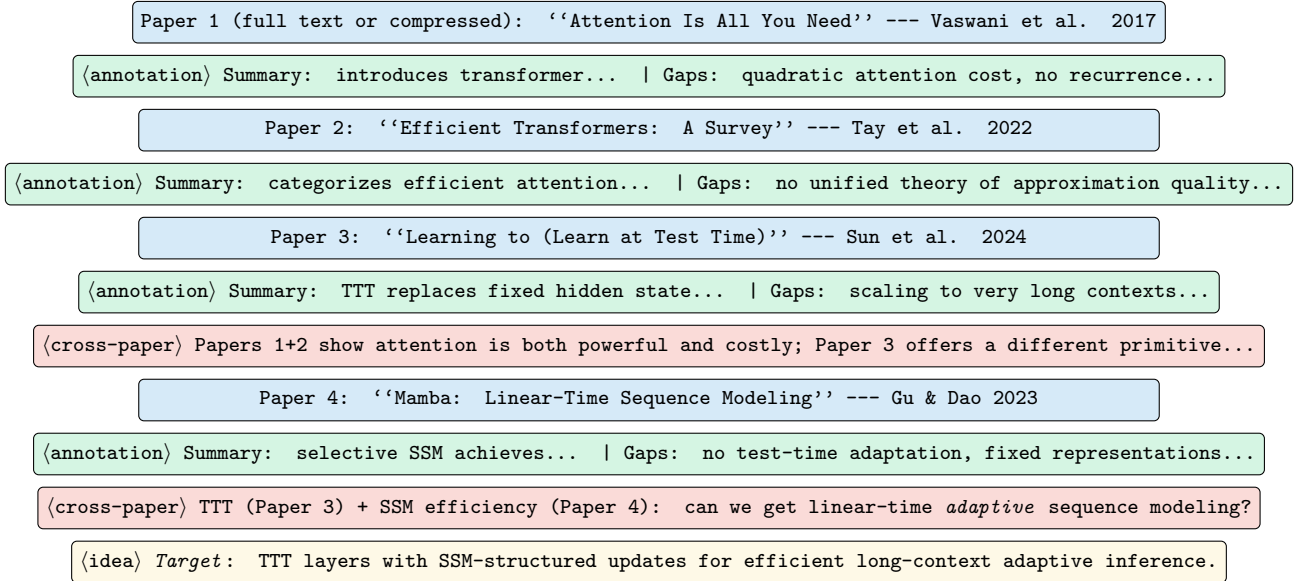


Figure 1: Megadoc stitching structure. Papers (blue) are interleaved with per-paper annotations (green), periodic cross-paper reasoning (red), and a final research idea target (gold). Annotations serve as latent thoughts between content segments. During TTT inner-loop training, the model processes the full sequence; at meta-evaluation time, accumulated annotations are retained in memory alongside the updated model to predict the idea.

3 Training: TTT for Taste

The key challenge is that real information streams are long and noisy—not every paper in a researcher’s reading history is relevant to the idea they eventually pursue. There are two approaches to handling irrelevant context: (a) *retrieval*: use a retriever to filter the stream before the model sees it, or (b) *selective absorption*: let the model consume everything and learn to selectively attend to what matters. I plan to test both, but TTT is specifically the answer to (b).

Inner loop (test-time). Given a megadoc sequence, the model processes papers and annotations via gradient-based self-supervised learning, minimizing cross-entropy on the content tokens. This is the TTT inner loop: the model’s parameters *adapt* to the specific literature review trajectory, absorbing the content of each paper and its annotations into its weights. Papers that are irrelevant to the eventual idea contribute less useful gradient signal; the model must learn to selectively update its representations based on what will matter downstream.

Outer loop (meta-learning). The meta-objective evaluates the *adapted* model: after the inner loop has processed the full paper sequence, I collect all accumulated annotations (summaries, gap analyses, cross-paper connections) and present them alongside the adapted model’s representations. The outer loss is cross-entropy on

the target research idea, given these annotations and the model state post-inner-loop. The meta-gradient flows through the inner loop, training the model to absorb information during reading in a way that maximally helps it generate good research directions afterward.

This two-loop structure directly addresses the needle-in-a-haystack problem: the outer loop rewards inner-loop updates that selectively encode the information relevant to the final idea, even when that information is buried in a long stream of partially-relevant papers.

4 Evaluation

Evaluating research taste is hard—it requires judging the quality of *questions*, not answers. I propose a hierarchy from cheap/automatic to expensive/definitive.

Intrinsic metrics. Cross-entropy on held-out research directions: given a literature review context, measure loss on the actual direction the author pursued. Plus auxiliary tasks: retrospective citation prediction (hold out one reference, predict and explain), gap identification (predict year- $Y+1$ directions from year- Y papers), and importance ranking (rank open questions by actual follow-up work generated).

LLM-as-Judge. A strong LLM scores generated directions on novelty, specificity, grounding, and feasibility—a scalable proxy I can run on thousands of outputs.

Generate → Execute → Judge. The full loop: (1) the model generates a concrete research idea with methodology and contribution statement; (2) a code agent implements it—runs experiments, generates figures, writes results; (3) I feed the output through a conference acceptance model trained on real venue decisions. This gives a scalar reward signal for RLHF-style fine-tuning: optimize taste by optimizing for directions that produce accepted papers.

Human expert evaluation. Blind comparison: domain experts rate generated vs. real research directions on novelty, feasibility, and importance, and I measure discriminability.

References

- Dong, J., Lyu, Z., & Ke, Q. (2024). Towards understanding evolution of science through language model series. *arXiv preprint arXiv:2409.09636*.
- Kim, K., Kotha, S., Choi, Y., Hashimoto, T., Haber, N., & Liang, P. (2025). Data-efficient pre-training by scaling synthetic megadocs. *arXiv preprint arXiv:2603.18534*.
- Yang, Z., Band, N., Li, S., Candès, E., & Hashimoto, T. (2024). Synthetic continued pretraining. *arXiv preprint arXiv:2409.07431*.
- Yang, Z., Zhang, A., Liu, H., Hashimoto, T., Candès, E., Wang, C., & Pang, R. (2025). Synthetic bootstrapped pretraining. *arXiv preprint arXiv:2509.15248*.